We Make You Shine
St. JOSEPH'S INSTITUTE OF TECHNOLOGY
(An Autonomous Institution)
St. Joseph's Group of Institutions
OMR, CHENNAI - 119

AWS WEB APPLICATION DEPLOYMENT USING AMAZON EC2

MINI PROJECT REPORT

Vijayaraghavan. S (312425205242)

Vishwa. V (312425205246)

Second Year

B.Tech Information Technology

Academic Year 2026–2027

TABLE OF CONTENTS

Sl. No	Contents
1	Introduction
2	Problem Statement
3	Scope/ Application of the project
4	Advantage of the project
5	System Requirements
6	Project Modules
7	Working of the project
8	Algorithm/ Deployment Process Used
9	Project Code
10	Output Screenshot
11	Conclusion
12	Future Enchancement
13	References

1. INTRODUCTION

Cloud computing has become an important technology for developing, hosting, and deploying modern web applications. Instead of depending completely on local computers or physical servers, cloud platforms provide computing resources through the internet.

Amazon Web Services (AWS) is one of the widely used cloud computing platforms. It provides services for computing, networking, storage, security, application deployment, and resource management.

The mini project “AWS Web Application Deployment Using Amazon EC2” was developed as part of my Cloud Engineering internship. The main objective is to understand how a web application can bedeployed on cloud infrastructure and made accessible through a public network.

In this project, an Amazon EC2 instance is used as the cloud server. The project uses an Ubuntu/Linux environment for managing the server. A simple HTML webpage was created and hosted using Python's

built-in HTTP server.

The webpage was configured to run on Port 8000. Appropriate AWS networking and security configurations were performed to allow access to the application.

The project also involved practical understanding of public and private IP addresses, Security Groups, VPC, subnets, route tables, Internet Gateway, NAT Gateway, Target Groups and Elastic

Load Balancer.

The project provided practical knowledge of cloud infrastructure and helped connect theoretical concepts with real-world application deployment.

2. PROBLEM STATEMENT

Traditional application deployment often depends on local computers or physical servers, creating limitations in accessibility, scalability, networking, and resource management.

A web application developed on a local computer may not be directly accessible outside the local environment. Additional configuration is required to make it available through a network.

The main problem addressed by this project is to deploy a web application on cloud infrastructure and make it accessible through a public endpoint.

The project aims to:

- Create a cloud-based computing environment.
- Deploy a web application on an EC2 instance.
- Configure a Linux server for application hosting.
- Run the application using Python HTTP server.
- Configure Port 8000 for application access.
- Configure Security Group rules.
- Understand public and private IP communication.
- Configure AWS networking components.
- Make the webpage accessible through a public endpoint.
- Test the deployed application through a web browser.

This provides a simple and practical method for understanding how cloud-based web applications are deployed and accessed.

3. SCOPE / APPLICATION OF THE PROJECT

3.1 Web Application Hosting

The project can be used to host simple web applications on Amazon EC2. The EC2 instance acts as a

virtual server where the application can be executed.

3.2 Cloud-Based Application Deployment

The project demonstrates how an application can be moved from a local development environment to a

cloud environment.

3.3 Educational Projects

Students can use similar cloud deployment methods for hosting academic projects and understanding

real-world cloud infrastructure.

3.4 Development and Testing

The EC2 environment can be used by developers to deploy and test web applications before using

them in larger environments.

3.5 Remote Application Access

Once the required networking and security configurations are completed, the deployed webpage can be

accessed remotely through a public endpoint.

3.6 Cloud Engineering Practice

The project provides practical exposure to EC2, Linux, SSH, IP addressing, Security Groups, VPC,

subnets, route tables, Internet Gateway, NAT Gateway, Target Groups and Load Balancing.

4. ADVANTAGES OF THE PROJECT

4.1 Cloud-Based Hosting

The application is hosted on AWS instead of depending only on a local computer.

4.2 Remote Accessibility

The deployed webpage can be accessed through a configured public endpoint.

4.3 Flexible Infrastructure

Amazon EC2 provides virtual computing resources that can be configured according to application requirements.

4.4 Practical Linux Experience

The project provides hands-on experience with an Ubuntu/Linux server, command-line operations, file management and server execution.

4.5 Network Configuration

The project provides practical knowledge about VPCs, subnets, routing, public and private IP addresses, and network connectivity.

4.6 Security Control

AWS Security Groups provide control over inbound and outbound network traffic. The required access for Port 8000 was configured.

4.7 Load Balancing

The project environment includes a Target Group and Elastic Load Balancer, providing practical exposure to directing application traffic.

4.8 Real-World Deployment Experience

The project provides experience with an actual cloud deployment workflow rather than only developing an application locally.

5. SYSTEM REQUIREMENTS

5.1 Hardware Requirements

- Laptop or desktop computer
- Minimum 4 GB RAM
- Stable internet connection
- Keyboard and mouse
- Sufficient storage space

5.2 Software Requirements

- Windows/Linux operating system
- Web browser
- AWS Management Console
- Ubuntu/Linux server
- SSH client
- Python
- HTML
- Terminal/Command Prompt

5.3 AWS Services and Components

Amazon EC2

Virtual cloud server for hosting the webpage.

Amazon VPC

Virtual networking environment for cloud resources.

Subnets

Separate public and private network sections.

Route Tables

Control network traffic routing.

Internet Gateway

Provides internet connectivity for public resources.

NAT Gateway

Provides outbound internet connectivity for private resources.

Security Groups

Control inbound and outbound traffic to EC2 instances.

Target Group

Registers application targets for load balancing.

Elastic Load Balancer

Provides an entry point for directing application traffic.

Python HTTP Server

Hosts the HTML webpage on Port 8000.

6. PROJECT MODULES

6.1 VPC Module

The VPC provides the basic virtual networking environment. The configured VPC contains separate

public and private subnets.

6.2 Subnet Module

Public and private subnets are distributed across Availability Zones. Public subnets provide network

access while private subnets provide network separation.

6.3 Route Table Module

Route tables control the movement of network traffic within the VPC.

6.4 Internet Gateway Module

The Internet Gateway provides connectivity between public VPC resources and the internet.

6.5 NAT Gateway Module

NAT Gateways provide outbound internet connectivity for private subnet resources.

6.6 EC2 Module

Amazon EC2 is the main computing component. The Ubuntu/Linux instance acts as the webpage

server.

6.7 SSH Module

SSH is used to connect to the Linux EC2 instance and execute commands.

6.8 Security Group Module

Security Groups control traffic to the EC2 instance, including the required Port 8000 access.

6.9 Web Application Module

A simple HTML webpage was created with the message “My First Project Is Successful”.

6.10 Python HTTP Server Module

Python's built-in HTTP server serves the HTML webpage on Port 8000.

6.11 Target Group Module

The Target Group identifies and manages application targets for the load-balancing configuration.

6.12 Load Balancer Module

The Elastic Load Balancer provides an endpoint for directing application traffic toward the configured

target.

7. WORKING OF THE PROJECT

The working of the project begins with the creation of the AWS networking environment.

First, a VPC is created to provide the virtual network for the project. Public and private subnets are

configured within the VPC.

Route tables are associated with the required subnets. The public route table is connected to an

Internet Gateway, allowing public resources to communicate with the internet.

NAT Gateways are configured for private network resources that require outbound connectivity.

After the networking environment is prepared, an Amazon EC2 instance is created using an

Ubuntu/Linux environment.

The EC2 instance is accessed through SSH. After establishing the connection, an HTML file named

index.html is created.

The webpage is then hosted using Python's built-in HTTP server.

```
python3 -m http.server 8000
```

The server starts listening on 0.0.0.0:8000. The Security Group is configured to allow the required traffic

through Port 8000.

The application can then be accessed through the configured public endpoint. A Target Group and Load

Balancer are also configured in the project environment.

The browser output displays “My First Project Is Successful”. The terminal shows successful HTTP

requests with status code 200.

Overall Workflow

1. Create VPC
2. Create Public and Private Subnets
3. Configure Route Tables
4. Configure Internet Gateway / NAT Gateway
5. Launch EC2 Instance
6. Connect through SSH
7. Create HTML Webpage
8. Start Python HTTP Server
9. Configure Port 8000
10. Configure Security Group
11. Configure Target Group / Load Balancer
12. Access Webpage through Browser
13. Verify Output

8. ALGORITHM / DEPLOYMENT PROCESS USED

The project does not use a traditional mathematical or machine-learning algorithm. It follows a cloud deployment and configuration process.

1. Start the AWS deployment process.
2. Create the required VPC.
3. Configure public and private subnets.
4. Configure route tables.
5. Attach the required Internet Gateway.
6. Configure NAT Gateway for private network access.
7. Launch an Ubuntu EC2 instance.
8. Configure the required Security Group.
9. Connect to the EC2 instance using SSH.
10. Create the HTML webpage.
11. Save the webpage as index.html.
12. Start the Python HTTP server.
13. Configure the server to use Port 8000.
14. Configure the required network access.
15. Configure the Target Group and Load Balancer.
16. Access the application through the configured endpoint.
17. Verify the webpage output.
18. Check server logs and HTTP response status.
19. Troubleshoot any configuration errors.
20. Confirm successful deployment.

9. PROJECT CODE

9.1 HTML Code

The project uses a simple HTML webpage.

<!DOCTYPE html>

<html>

<head>

<title>My First Project</title>

</head>

<body>

<h1>My First Project Is Successful</h1>

</body>

</html>

The HTML file is saved as index.html.

9.2 Python HTTP Server

The webpage is hosted using Python's built-in HTTP server.

```
python3 -m http.server 8000
```

This starts the HTTP server on Port 8000 and serves the files present in the current directory.

9.3 Server Output

```
Serving HTTP on 0.0.0.0 port 8000
```

The terminal records incoming requests and their HTTP response status. A successful request is shown

with GET / HTTP/1.1 200. Status code 200 indicates that the requested webpage was successfully

served.

10. OUTPUT SCREENSHOT

The final output of the project is a successfully deployed webpage. The browser displays “My First Project Is Successful”. The webpage is accessed through the AWS-configured endpoint on Port 8000.

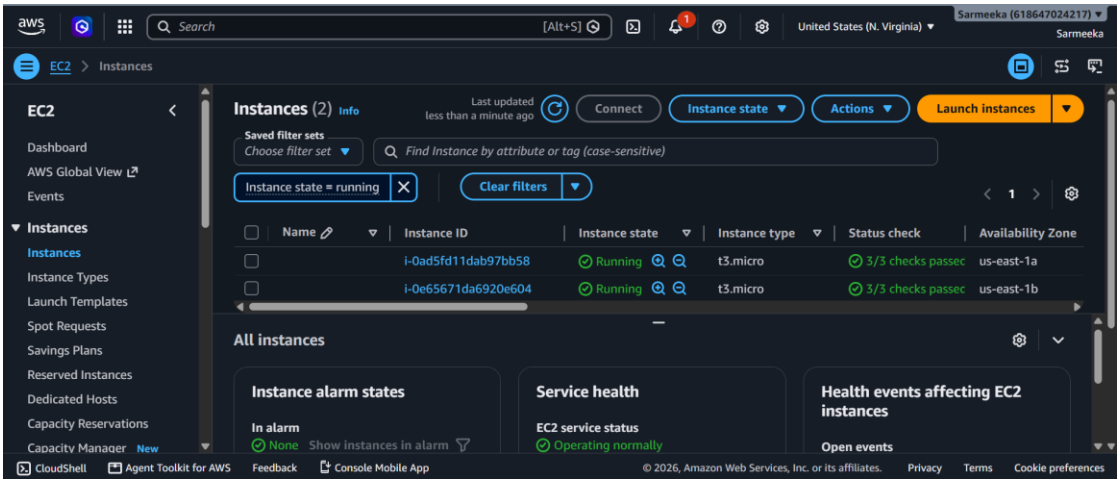


Figure 10.1 – Amazon EC2 Instances

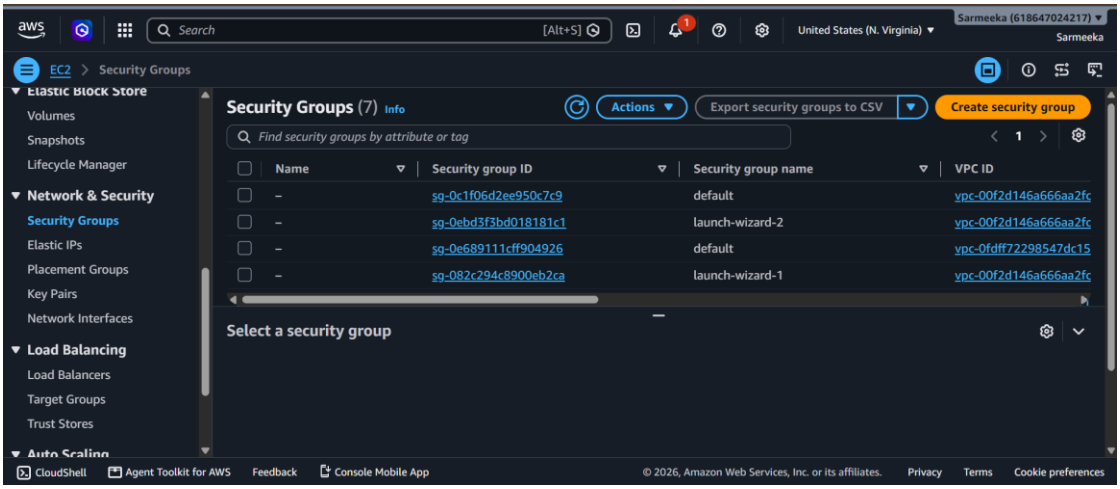


Figure 10.2 – AWS Security Groups



Figure 10.3 – VPC Resource Map

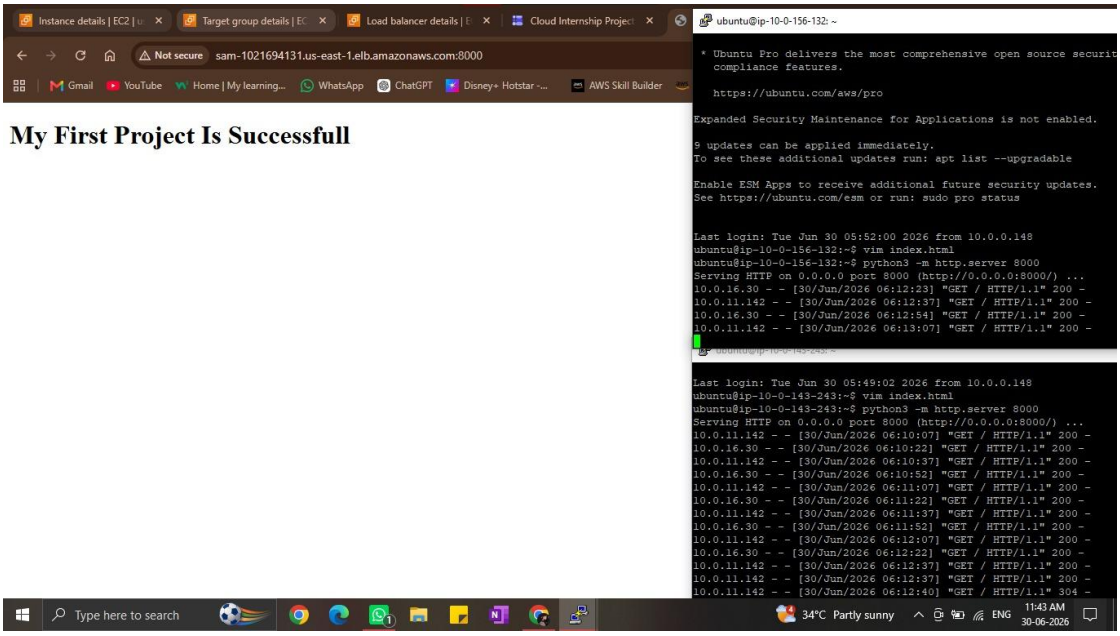


Figure 10.4 – Deployed Webpage and Python HTTP Server

11. CONCLUSION

The AWS Web Application Deployment Using Amazon EC2 mini project provided practical experience in cloud computing and web application deployment.

The project demonstrated how a simple HTML webpage can be hosted on an AWS EC2 instance using an Ubuntu/Linux environment. The webpage was served using Python's built-in HTTP server and configured to run on Port 8000.

The project provided practical understanding of VPCs, public and private subnets, route tables, Internet Gateway, NAT Gateway, public and private IP addresses, and Security Groups.

It also provided exposure to Target Groups and Elastic Load Balancing for application access through an AWS endpoint.

The project helped bridge the gap between theoretical knowledge and practical implementation and improved understanding of Linux server management, cloud networking, AWS infrastructure, application deployment, troubleshooting and technical documentation.

Overall, the project provided a strong foundation for understanding Cloud Computing, Cloud Engineering and DevOps.

12. FUTURE ENHANCEMENTS

12.1 Database Integration

A database can be integrated to store application information and user data.

12.2 Domain Name Integration

A custom domain name can be connected to the application instead of a cloud-generated endpoint.

12.3 HTTPS Configuration

HTTPS can be implemented to provide encrypted communication.

12.4 Advanced Load Balancing

Load balancing can be expanded across multiple EC2 instances.

12.5 Auto Scaling

Auto Scaling can increase or decrease EC2 instances according to application demand.

12.6 Application Monitoring

Cloud monitoring and logging can be integrated to track application health and errors.

12.7 CI/CD Integration

A CI/CD pipeline can automatically deploy application updates.

12.8 Improved Web Application

The simple HTML page can be expanded into a dynamic application with backend, database and authentication.

12.9 Improved Security

Additional security controls such as HTTPS, restricted Security Group rules, IAM permissions and secure key management can be implemented.

13. REFERENCES

1. Amazon Web Services – Amazon EC2 Documentation
2. Amazon Web Services – Amazon VPC Documentation
3. Amazon Web Services – Security Groups Documentation
4. Amazon Web Services – Elastic Load Balancing Documentation
5. Python Documentation – HTTP Server
6. HTML Documentation
7. AWS Skill Builder Learning Resources